ENGINEERING REBOOT 2026

Day 34 Field Manual

Version 1.0

Week 07 — Software Architecture & Object-Oriented Programming

---

Mission Brief

Today marks an important transition in your engineering development.

Up to this point, you have learned how to create classes, instantiate objects, encapsulate data, and compose multiple classes into a larger system.

Today you will learn an equally important professional engineering skill:

Improving existing software without changing its behavior.

Professional software engineers spend considerably more time improving existing systems than creating completely new ones.

The goal is not to make software "fancier."

The goal is to make software:

- easier to understand
- easier to extend
- easier to maintain
- easier to test
- easier to debug

Engineering Context

Many junior programmers believe that writing code is the hard part.

Professional engineers know something different.

Writing Version 1 is usually the easy part.

Maintaining Version 10 is the real challenge.

Today's work introduces disciplined refactoring:

- improving design
- reducing duplication
- increasing readability
- separating responsibilities
- preserving behavior

without introducing unnecessary complexity.

Learning Objectives

Technical Objectives

By the end of today you should be able to:

- recognize procedural code that should become a class
- move related behavior into appropriate methods
- identify duplicated logic
- improve naming
- simplify interfaces
- separate responsibilities between classes
- preserve existing behavior while improving design

Engineering Objectives

Develop the discipline to:

- improve software incrementally
- avoid unnecessary rewrites
- recognize "good enough" designs
- understand that readability is an engineering feature
- perform safe refactoring

Core Concepts

Today's concepts include:

- Refactoring
- Code smells
- Single Responsibility Principle (introduction)
- Cohesion
- Coupling (introduction)
- Separation of Concerns
- Encapsulation review
- Maintaining external behavior

Exercises

Exercise 1

Identifying Refactoring Opportunities

Study a procedural program.

Identify:

- duplicated code
- long functions
- mixed responsibilities
- poor naming
- global data
- repeated logic

Do not modify the program yet.

Simply annotate opportunities.

Deliverable:

`refactoring_notes.md`

Exercise 2

Extract Responsibilities

Take one procedural function.

Convert it into a method inside an appropriate class.

Example:

Instead of

```
calculate_total(...)
```

consider

```
shopping_cart.calculate_total()
```

Focus on responsibility rather than syntax.

Exercise 3

Improve Naming

Review one existing program.

Rename variables, methods, and classes where appropriate.

Examples:

Instead of

```
x
temp
process()
```

prefer

```
employee_count
transaction_total
calculate_payroll()
```

Behavior must remain unchanged.

Exercise 4

Reduce Duplication

Find repeated logic.

Extract reusable helper methods.

Avoid copy-and-paste implementations.

Mini Project

Object-Oriented Refactoring

Rather than building an entirely new application, today's project is to improve one you already wrote.

Recommended candidates include:

- Employee Reporting System
- Employee Directory
- Registration System
- Library Management System (if you intentionally left opportunities for improvement)

Choose **one**.

Requirements

Without changing user-visible behavior:

- improve organization
- simplify methods
- reduce duplication
- improve naming
- strengthen encapsulation where appropriate
- move related behavior into classes
- remove unnecessary code

Do **not** add new features.

Do **not** redesign the application.

Do **not** optimize prematurely.

The purpose is disciplined refactoring.

Suggested Refactoring Checklist

Review whether the application can benefit from:

- smaller methods
- clearer class responsibilities
- improved naming
- removal of duplicate code
- simplified conditional logic
- improved comments (where necessary)
- better organization of helper methods

Remember:

Not every suggestion must be implemented.

Apply engineering judgment.

Validation Matrix

You have completed today's objectives when:

✓ Existing functionality still works

✓ No new bugs introduced

✓ Code readability improved

✓ Responsibilities are clearer

✓ Duplication reduced

✓ Public behavior unchanged

Evidence Checklist

Capture:

- source code
- program execution
- before/after screenshots (optional but encouraged)
- Git operations

Required artifacts:

```
Refactored-Code.png
Refactored-Output.png
Git-Operations.png
```

Engineering Review

Before closing Day 34, ask yourself:

- Did I improve readability?
- Did I avoid unnecessary complexity?
- Did I preserve behavior?
- Did I make future maintenance easier?
- Would another engineer understand this code more quickly?

If the answer is yes, today's objective has been achieved.

Notes.md Guidance

Record:

- refactoring opportunities discovered
- improvements made
- decisions intentionally deferred
- lessons learned about maintainability

Journal.md Guidance

Reflect on:

- How difficult was it to improve existing code?
- Did you discover weaknesses in your earlier design?
- Which changes produced the greatest improvement with the least effort?
- How did today's work change your view of writing versus maintaining software?

Completion Checklist

- Exercise 1 completed
- Exercise 2 completed
- Exercise 3 completed
- Exercise 4 completed
- Mini Project completed
- Evidence captured
- Notes.md written
- Journal.md written
- Git committed
- Git pushed
- Repository verified

Preview of Day 35

Day 35 concludes Week 7 with **Week Integration**.

Rather than introducing new language features, you will build a small multi-class application that integrates everything learned during the week:

- Classes
- Constructors
- Instance methods
- Encapsulation
- Composition
- Refactoring principles

The objective is to produce a clean, maintainable object-oriented design while remaining appropriately scoped for the Engineering Reboot.